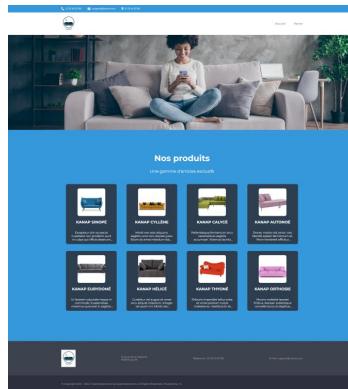


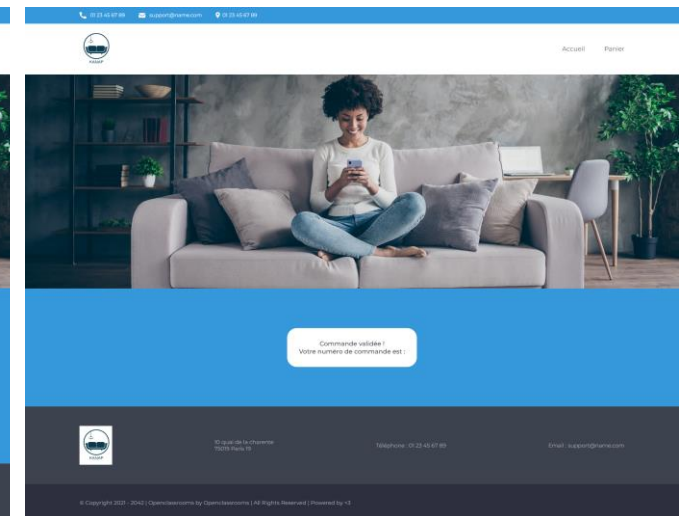
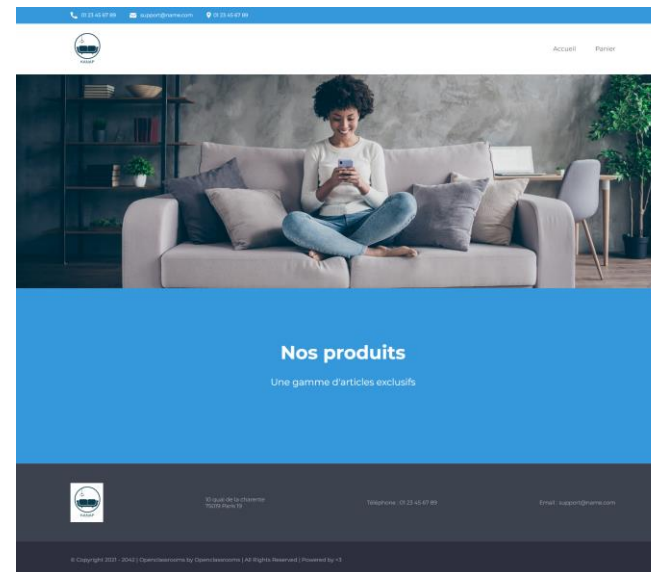
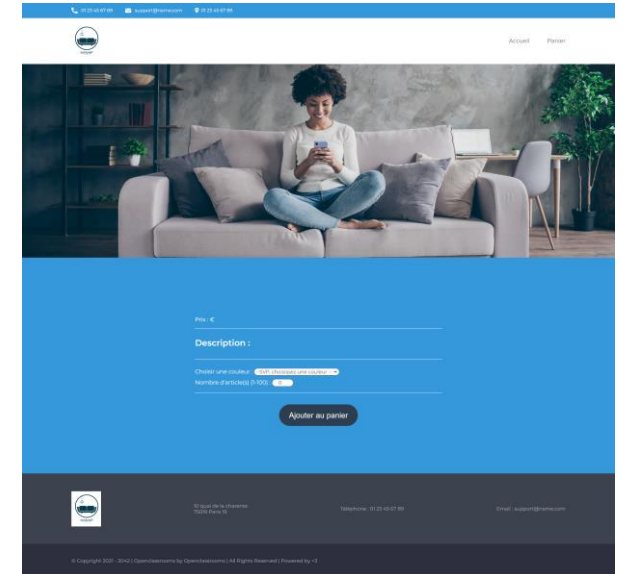
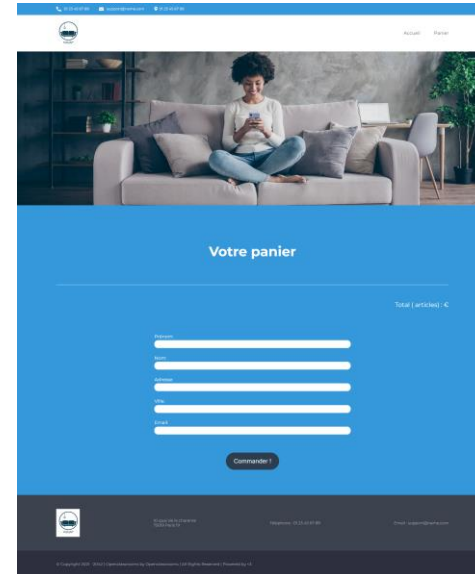


Site e-commerce de canapés exclusifs

Kanap

Présentation & mission

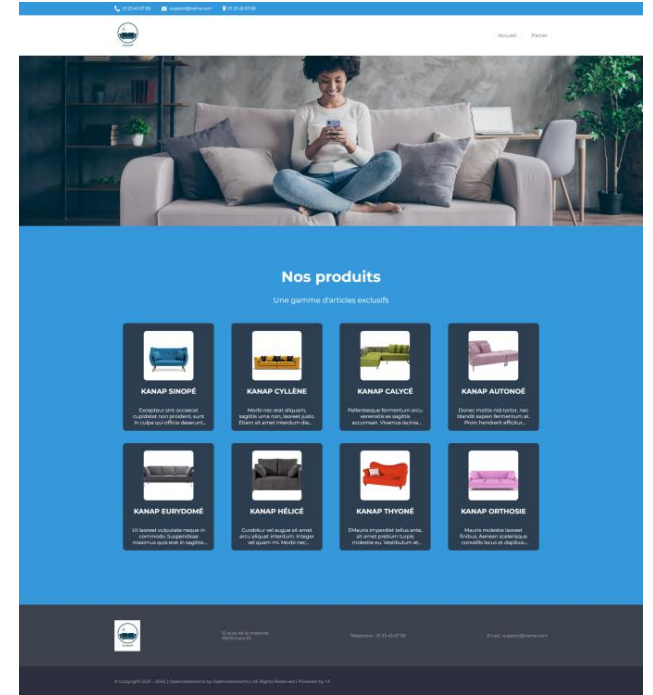
- Kanap, une marque de canapés avec boutique physique
- Projet : une plateforme de e-commerce
- Intégrer ces éléments dynamiquement grâce à JS et l'API.
- Mettre en place un plan de test d'acceptation à partir du template fourni.
- Spécifications :
 - Utilisation de code JavaScript pur. Pas de framework ou librairie autorisé.
 - Indentation du code source et insertions de commentaires en début de fonction
 - L'application web sera composée de 4 pages





Page n° 1 - Accueil

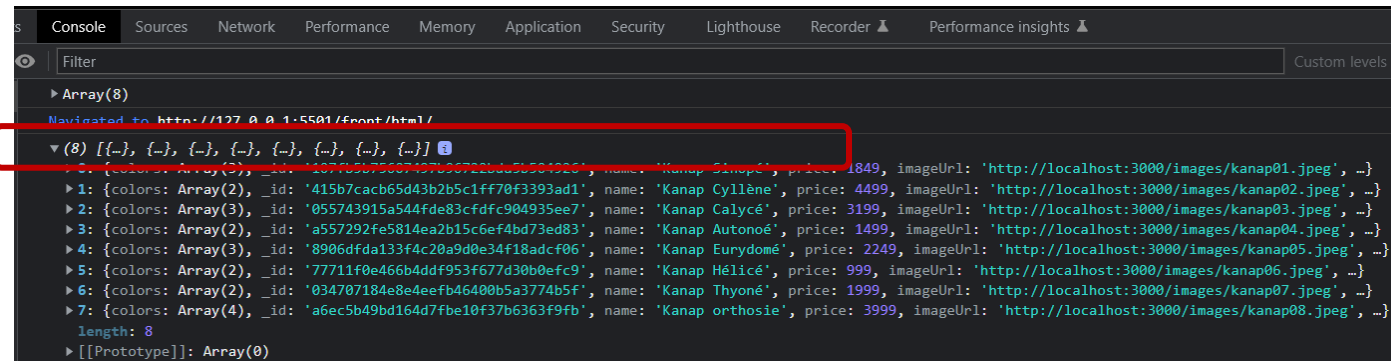
Une page d'accueil montrant (de manière dynamique) tous les articles disponibles à la vente.



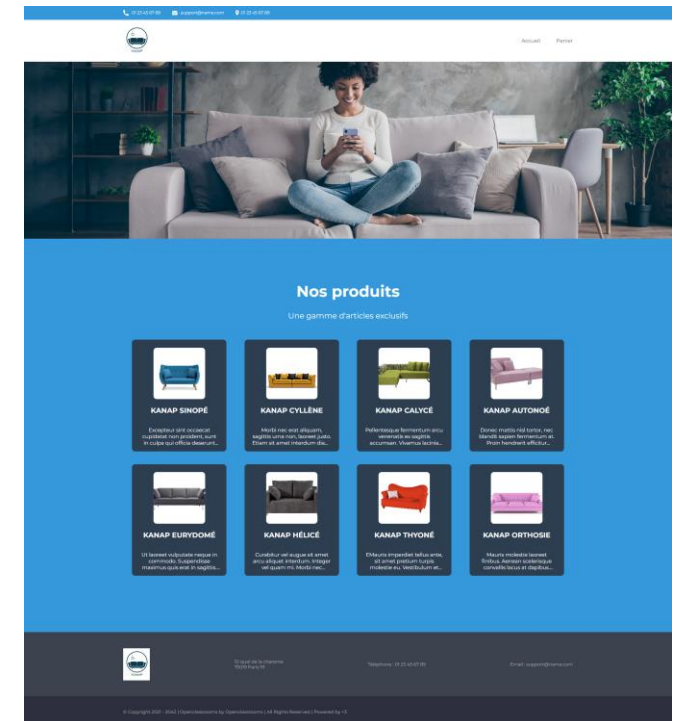
Affichage de tous les produits

- Requête l'API pour lui demander l'ensemble des produits

```
fetch("http://localhost:3000/api/products")  
  .then(function(response){  
    response.json()  
    .then(function(products){
```



```
Array(8)  
Navigated to http://127.0.0.1:5501/front/html/  
▼ (8) [{"id": "415b7cacb65d43b2b5c1ff70f3393ad1", "name": "Kanap Cyllène", "price": 4499, "imageUrl": "http://localhost:3000/images/kanap02.jpeg", "colors": Array(2)}, {"id": "055743915a544fde83cfdfc904935ee7", "name": "Kanap Calycé", "price": 3199, "imageUrl": "http://localhost:3000/images/kanap03.jpeg", "colors": Array(3)}, {"id": "a557292fe5814ea2b15c6ef4bd73ed83", "name": "Kanap Autoané", "price": 1499, "imageUrl": "http://localhost:3000/images/kanap04.jpeg", "colors": Array(2)}, {"id": "8906dfda133f4c20a9d0e34f18adc06", "name": "Kanap Eurydomé", "price": 2249, "imageUrl": "http://localhost:3000/images/kanap05.jpeg", "colors": Array(3)}, {"id": "77711f0e466b4dd953f677d30b0efc9", "name": "Kanap Héliacé", "price": 999, "imageUrl": "http://localhost:3000/images/kanap06.jpeg", "colors": Array(2)}, {"id": "034707184e8e4eefb46400b5a3774b5f", "name": "Kanap Thyoné", "price": 1999, "imageUrl": "http://localhost:3000/images/kanap07.jpeg", "colors": Array(4)}, {"id": "a6ec5b49bd164d7f8e10f37b6363f9fb", "name": "Kanap orthosie", "price": 3999, "imageUrl": "http://localhost:3000/images/kanap08.jpeg", "colors": Array(4)}]  
length: 8  
[[Prototype]]: Array(0)
```



Redirection page produit

- Pour chaque produit, il faudra afficher l'image de celui-ci, ainsi que son nom et le début de sa description.
- En cliquant sur le produit, l'utilisateur sera redirigé sur la page du produit pour consulter celui-ci plus en détail.

			
KANAP SINOPE	KANAP CYLLÈNE	KANAP CALYCÉ	KANAP AUTONOÉ
Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt...	Morbi nec erat aliquam, sagittis urna non, laoreet justo. Etiam sit amet interdum dia...	Pellentesque fermentum arcu venenatis ex sagittis accumsan. Vivamus lacinia...	Donec mattis nisl tortor, nec blandit sapien fermentum at. Proin hendrerit efficitur...



URLSearchParams

- C'est grâce à cette notion que la page Produit va pouvoir récupérer l'id du produit ayant été cliqué sur la page d'accueil
- requêter l'API dans le but de récupérer les différentes informations du produit en question.

```
// Extraire ID avec URLSearchParams
const urlParams = (new URL(location)).searchParams;
const idPage = urlParams.get("id");
```

```
const imageContainer = document.getElementsByClassName("item_img")[0]
```

```
// Récupérer les produits avec les id associés
fetch(`http://localhost:3000/api/products/${idPage}`)
  .then((response) => {
    response.json() // ma réponse (la liste des produits) en "response.json" (cad mon article au format JSON)
    .then((product) => {
      console.log({product});
      const img = document.createElement("img"); // là je crée juste la balise img
      img.src = product.imageUrl; // on set l url de l image dans la balise qu on vient de créer
      img.alt = product.altTxt; // on set l alt text de l image dans la balise qu on vient de créer
      imageContainer.appendChild(img); // on injecte la balise qu on a créé dans le container
```

```
▼ {idPage: '034707184e8e4eeffb46400b5a3774b5f'} ⓘ product.js:16
  idPage: "034707184e8e4eeffb46400b5a3774b5f"
  ► [[Prototype]]: Object
```

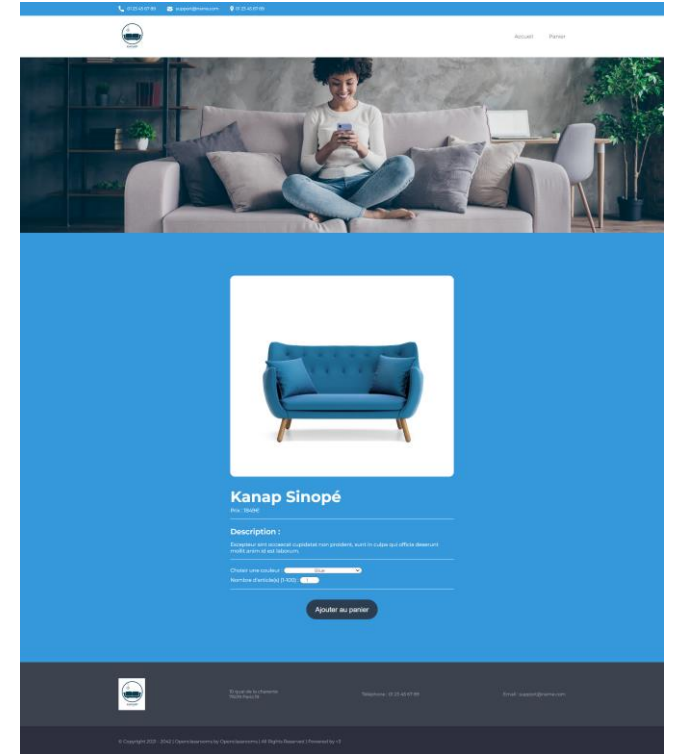




Page n° 2 - Produit

Une page “produit” qui affiche (de manière dynamique) les détails du produit sur lequel l'utilisateur a cliqué depuis la page d'accueil.

Depuis cette page, l'utilisateur peut sélectionner une quantité, une couleur, et ajouter le produit à son panier.



Récupérer et afficher un produit



Utilisation de l'API afin de récupérer un seul et unique produit, et non pas l'ensemble des produits

```
// Récupérer les produits avec les id associés
fetch(`http://localhost:3000/api/products/${idPage}`)
  .then((response) => {
    response.json() // ma réponse (la liste des produits) en "response.json"
```

```
const selectColors = document.getElementById("colors")
const selectQuantity = document.getElementById("quantity")
const selectNames = document.getElementById("title");
```

```
.then((product) => {
  console.log({product});
  const img = document.createElement("img"); // là je crée juste la balise img
  img.src = product.imageUrl; // on set l url de l image dans la balise qu on vient de créer
  img.alt = product.altTxt; // on set l alt text de l image dans la balise qu on vient de créer
  imageContainer.appendChild(img); // on injecte la balise qu on a créé dans le container
```

```
// Récupérer et afficher choix couleurs
for (let color of product.colors) {
  console.log(color)
  const option = document.createElement("option")
  option.value = color
  option.innerHTML = color
  selectColors.appendChild(option)
```

Un menu déroulant permettant à l'utilisateur de choisir une option de personnalisation

Conditions pour que l'ajout au panier soit valide

```
function addToCart(id, color, quantity){
  const products = kanapsSavedInLocalStorage();
  if (products[id]){
    if (products[id][color]){
      products[id][color] = parseInt(products[id][color]) + parseInt(quantity);
      // update en recuperant l'ancienne quantité
    } else {
      products[id][color] = parseInt(quantity);
    }
  }
  if (!products[id]){
    products[id] = {
      [color]:parseInt(quantity)
    }
  }
  updateKanapsSavedInLocalStorage(products)
}
```

Fonctions

```
function canAddToCart(){
  if (selectQuantity.value < 1 || selectQuantity.value >=100){
    alert('Veuillez choisir une quantité comprise entre 1 et 100');
    return false
  }
  if (selectColors.value == null || selectColors.value == ""){
    alert('Veuillez choisir une couleur');
    return false
  }
  return true;
}
```

```
// écouter EVT et récupérer sélection pour transférer au panier
btnAddProduct.addEventListener("click", function(event){
  if(canAddToCart()){
    addToCart(idPage, selectColors.value, selectQuantity.value)
    alert("L'article a bien été ajouté au panier")
  }
})
```

Ecouteur d'évènement



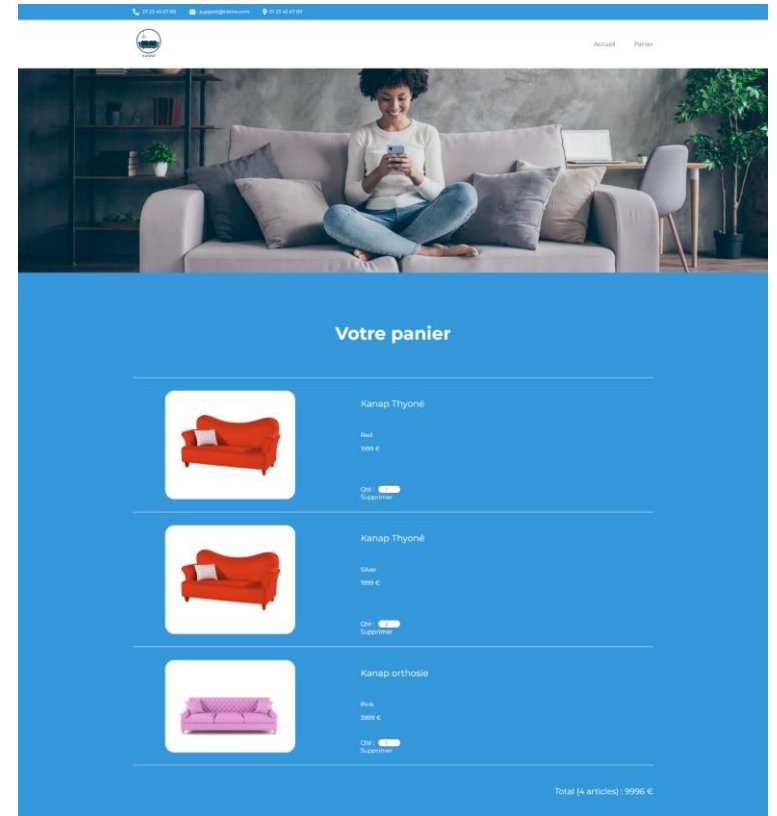
KANAP



Page n° 3 - Panier

Un résumé des produits dans le panier, le prix total et la possibilité de modifier la quantité d'un produit sélectionné ou bien de supprimer celui-ci.

Un formulaire permettant de passer une commande. Les données du formulaire doivent être correctes et bien formatées avant d'être renvoyées au back-end.





Page n° 3 - Panier

Un résumé des produits dans le panier

LocalStorage

Utilisation du localStorage

- Définir l'emplacement de sauvegarde des données dans le localStorage pour pouvoir accéder à cet objet depuis la page Panier
- Lorsqu'on ajoute un produit au panier, si celui-ci n'était pas déjà présent dans le panier, on ajoute un nouvel élément.
- (transformer objet JSON en string pour pouvoir le stocker)

```
// Mettre à jour les produits stockés dans le localStorage (transformer en string)  
function updateKanapsSavedInLocalStorage(products){  
  storageAccess.setItem("kanapSaved", JSON.stringify(products))  
}
```

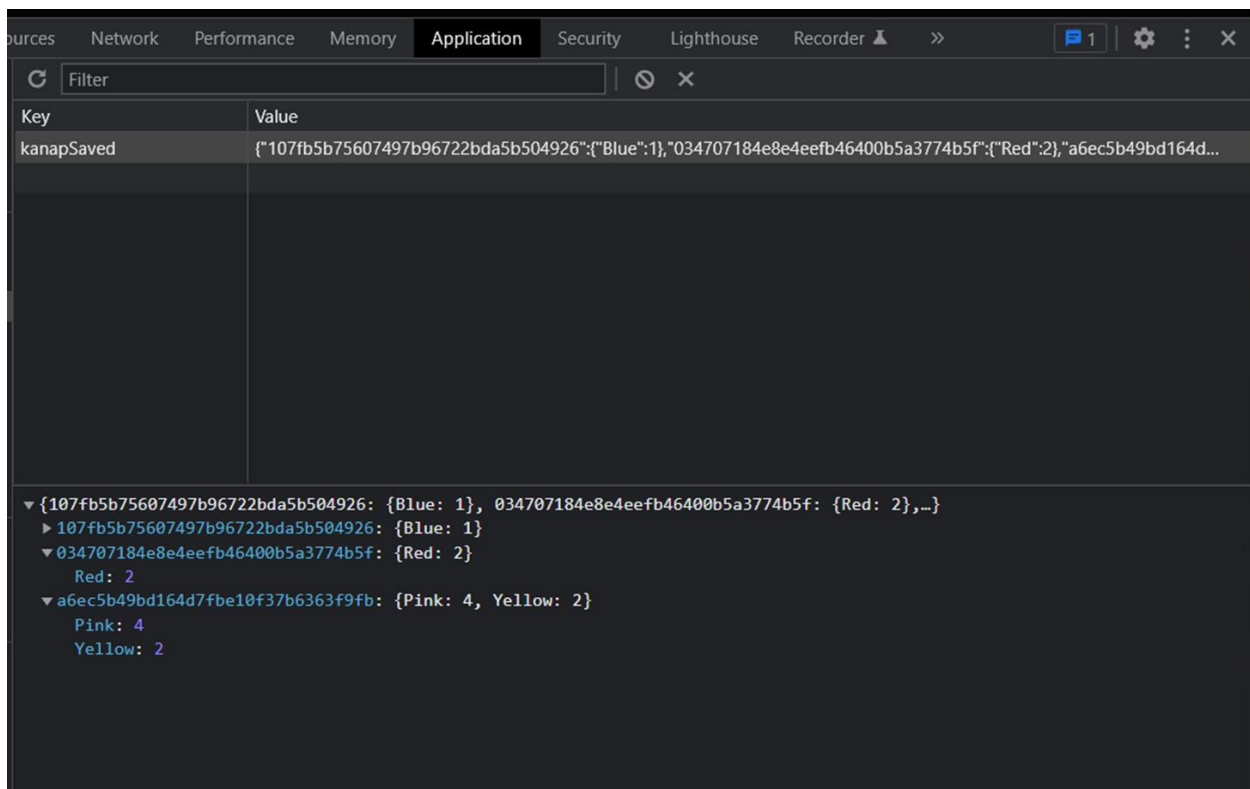
Récupérer le panier via localStorage

- Depuis la page Panier, récupérer le panier qui est un objet qui comprend trois choses :
 - l'id du produit ;
 - la quantité du produit ;
 - la couleur du produit.

```
function kanapsSavedInLocalStorage(){  
  const products = storageAccess.getItem("kanapSaved");  
  if (!products){  
    return {}  
  }  
  // Récupérer produits sauvegardés (format objet)  
  return JSON.parse(products)  
}
```

Un résumé des produits dans le panier

- le panier est un objet qui comprend trois choses :
 - l'id du produit ; la quantité du produit et la couleur du produit.



Votre panier



Kanap Héricé

Grey

999 €

Qté :

[Supprimer](#)



Kanap Calycé

Green

3199 €

Qté :

[Supprimer](#)



Kanap Calycé

Red

3199 €

Qté :

[Supprimer](#)

Total (7 articles) : 20193 €

Modifications dans la page Panier

la méthode `Element.closest()` permet de cibler le produit que l'on souhaite supprimer ou modifier grâce à son identifiant et sa couleur

Fonction

```
function modifyQuantityInLocalStorage(productId, productColor, productQuantity){
  let products = kanapsSavedInLocalStorage();
  if(productQuantity===0){
    removeProductFromLocalStorage(productId, productColor)
  }
  else{
    products[productId][productColor] = productQuantity;
    updateKanapsSavedInLocalStorage(products)
  }
}
```

Fonctions

```
function removeProductFromLocalStorage (productId, productColor){
  //Récupérer les éléments qui ont été changés grâce à l'eventListener dans cart
  let products = kanapsSavedInLocalStorage();
  let product=products[productId][productColor]
  if (product){
    if(Object.keys(products[productId]).length > 1){
      delete products[productId][productColor];
    } else {
      delete products[productId];
    }
  }

  updateKanapsSavedInLocalStorage(products)
}
```

```
const quantityBtns = document.getElementsByClassName("itemQuantity");
Object.values(quantityBtns).forEach(quantityBtn => {
  quantityBtn.addEventListener("change", function(event){
    /* Récupérer les informations via les attributs de l'article le plus proche
    pour les envoyer dans la fonction de modification.*/
    let article = quantityBtn.closest("article");
    console.log(article)
    let productId = article.getAttribute("data-id");
    console.log(productId)
    let productColor = article.getAttribute("data-color");
    let productQuantity=quantityBtn.value;

    modifyQuantityInLocalStorage (productId, productColor, parseInt(productQuantity));
    location.reload();
  })
})
```

Ecouleurs d'évènement

```
const removeBtns = document.getElementsByClassName("deleteItem");
Object.values(removeBtns).forEach(removeBtn => { //foreach permet de récupérer l'index
  removeBtn.addEventListener("click", function(){
    /* Récupérer les informations via les attributs de l'article le plus proche
    pour les envoyer dans la fonction de suppression.*/
    let article = removeBtn.closest("article");
    console.log(article)
    let productId = article.getAttribute("data-id");
    console.log(productId)
    let productColor = article.getAttribute("data-color");
    console.log(productColor)

    if(confirm("Voulez vous supprimer?")){
      removeProductFromLocalStorage(productId, productColor);
      location.reload()
    }
  })
})
```

Suppression d'un produit depuis le panier

**NE PAS PRENDRE QU'ID
SINON SUPPRESSION TOUS
LES PROPRIO PRODUITS**

```
function removeProductFromLocalStorage (productId, productColor){  
  //Récupérer les éléments qui ont été changés grâce à l'eventListener dans cart  
  let products = kanapsSavedInLocalStorage();  
  let product=products[productId][productColor]  
  if (product){  
    if(Object.keys(products[productId]).length > 1){  
      delete products[productId][productColor];  
    } else {  
      delete products[productId];  
    }  
  }  
  
  updateKanapsSavedInLocalStorage(products)  
}
```

```
const removeBtns = document.getElementsByClassName("deleteItem");  
Object.values(removeBtns).forEach(removeBtn => { //foreach permet de récupérer l'index  
  removeBtn.addEventListener("click", function(){  
    /* Récupérer les informations via les attributs de l'article le plus proche  
    pour les envoyer dans la fonction de suppression.*/  
    let article = removeBtn.closest("article");  
    console.log(article)  
    let productId = article.getAttribute("data-id");  
    console.log(productId)  
    let productColor = article.getAttribute("data-color");  
    console.log(productColor)  
  
    if(confirm("Voulez vous supprimer?")){  
      removeProductFromLocalStorage(productId, productColor);  
      location.reload()  
    }  
  })  
})
```

Ecouteurs d'évènement

Affichage dynamique du prix et de la qté

Object.entries

```
for(let [color, quantity] of Object.entries(colors)){
```

Prix & quantité

```
const totalQtyLabel = document.getElementById("totalQuantity");  
let nbproducts = 0;  
let totalprice = 0;  
const totalPriceLabel = document.getElementById("totalPrice");
```

```
console.log(nbproducts)  
totalprice+=product.price * parseInt(quantity);  
totalPriceLabel.innerHTML = totalprice;
```

InnerHTML

```
for(let [color, quantity] of Object.entries(colors)){  
  
  nbproducts += parseInt(quantity);  
  totalQtyLabel.innerHTML = nbproducts;  
  
  console.log(nbproducts)  
  totalprice+=product.price * parseInt(quantity);  
  totalPriceLabel.innerHTML = totalprice;  
  
  cartContainer.innerHTML += `  
    <article class="cart_item" data-id="${product._id}" data-color="${color}">  
      <div class="cart_item_img">  
          
      </div>  
      <div class="cart_item_content">  
        <div class="cart_item_content_description">  
          <h2>${product.name}</h2>  
          <p>${color}</p>  
          <p>${product.price} €</p>  
        </div>  
        <div class="cart_item_content_settings">  
          <div class="cart_item_content_settings_quantity">  
            <p>Qté :</p>  
            <input type="number" class="itemQuantity" name="itemQuantity" min="0" max="100" value="${quantity}">  
          </div>  
          <div class="cart_item_content_settings_delete">  
            <p class="deleteItem">Supprimer</p>  
          </div>  
        </div>  
      </div>  
    </article>  
  `;  
}
```



Page n° 3 - Panier

Un formulaire permettant de passer une commande

Les inputs utilisateurs doivent être vérifiés et validés

```
for(let input of allInputs){
  let RegexForInput;// permettra de définir les types de regex
  if (input.type === "submit"){
    break;
  }
  if (input.type === "text"){
    RegexForInput = /^[a-zA-Z0-9\s, '-]*$/;
  }
  if (input.type === "email"){
    RegexForInput = /^[w.-]+@[w-]+\.\w{3,6}$/;
  }
  if(input.value === ""){
    input.style.border = "1px solid red";
    input.nextElementSibling.innerHTML = "ce champs doit être complété";
  }
  else if(input.value.length<3){
    input.style.border = "1px solid red";
    input.nextElementSibling.innerHTML = "ce champs doit comporter min 3 caractères";
  }
  else if(input.value.match(RegexForInput) === null){
    input.style.border = "1px solid red";
    input.nextElementSibling.innerHTML = "champs invalide";
  }
  else {
    input.style.border = "1px solid green";
    input.nextElementSibling.innerHTML = "";
    valid++
  }
}
```

```
const orderForm = document.getElementsByClassName("cart__order__form")[0];
const allInputs = orderForm.elements;

document.querySelector("#order").addEventListener("click", (event) => {
  event.preventDefault();

  let valid = 0;

  if(valid > 0){
    // créer un bouton de validation
```

Ecouteur d'évènement

Vérification du
format des inputs
via les Regex

L'utilisateur doit pouvoir valider sa commande

objet contact

- Constituer un objet contact (à partir des données du formulaire)
- Pour les routes POST, l'objet contact envoyé au serveur doit contenir les champs firstName, lastName, address, city et email.

```
if(valid===5){  
  // créer objet contact + product avant envoi.  
  const contact = { // objet contact en mettant les informations de chaque champ du formulaire  
    firstName : allInputs[0].value,  
    lastName : allInputs[1].value,  
    address : allInputs[2].value,  
    city :allInputs[3].value,  
    email : allInputs[4].value  
  }  
}
```

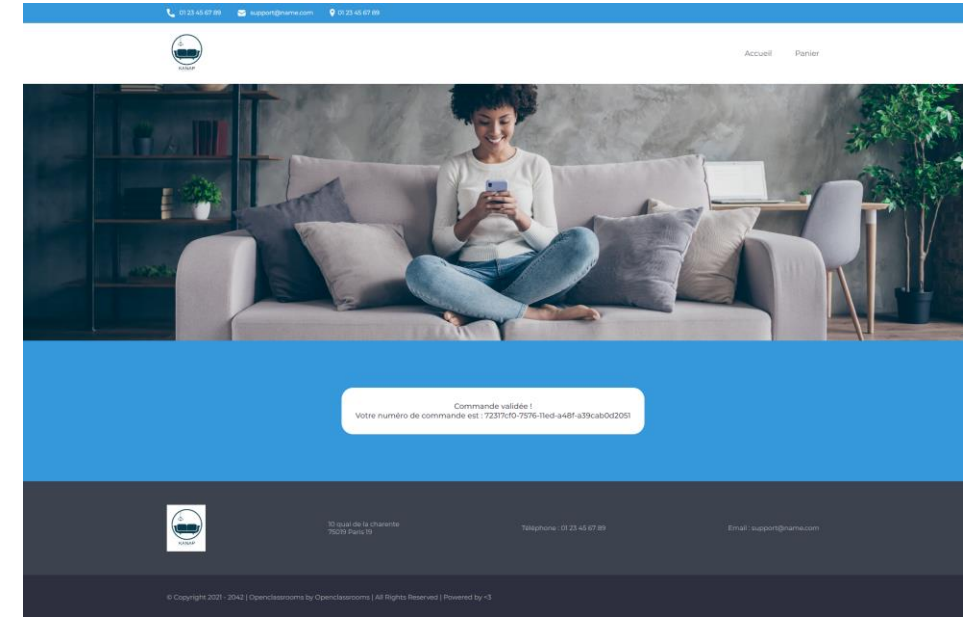
tableau de produits

- Constituer un tableau de produits
- Le tableau des produits envoyé au back-end doit être un array de strings product-ID.

```
const products = []; // tableau "products" qui va contenir les id des produits du panier pour ensuite les envoyer.
```

```
for(let [id, colors] of Object.entries(productsInCart)){ // boucler sur les prod du panier.  
  products.push(id) // pousser dans array product.  
}
```

```
    ▶ {firstName: 'chaimaa', lastName: 'asri', address: '15 avenue FERDINAND BUISSON', city: 'PARIS', email: 'chaimaa.asri@gmail.com'} cart.js:135  
    ▶ (3) ['107fb5b75607497b96722bda5b504926', '034707184e8e4eefb46400b5a3774b5f', 'a6ec5b49bd164d7fbe10f37b6363f9fb'] cart.js:145  
    Navigated to http://127.0.0.1:5501/front/html/confirmation.html?orderId=f753b770-7bd6-11ed-8073-c34f9ef1415e  
    >
```



Page n° 4 - Confirmation

Validation des données & Requête POST

Validation des données

- Les types de ces champs et leur présence doivent être validés avant l'envoi des données au serveur.

```
const orderForm = document.getElementsByClassName("cart__order__form")[0];
const allInputs = orderForm.elements;
```

```
document.querySelector("#order").addEventListener("click", (event) => {
  event.preventDefault();
```

```
    let valid = 0;
```

```
    if(valid===5){
      // creer objet contact + product avant envoi.
```

Requête POST

- Effectuer une requête POST sur l'API et récupérer l'identifiant de commande dans la réponse de celle-ci.

```
fetch('http://localhost:3000/api/products/order',{ // fetch pour l'envoi tableau.
  method : "POST",
  headers : {"Content-type":"application/json;charset=UTF-8"}, // définit les entêtes
  body : JSON.stringify({contact, products})// corps de la requete (l'objet contact + products)
})
.then (function(response){
  response.json()
  .then(function(orderInfo){
    // rediriger vers page confirmation avec parametre dans l'URL (en ajoutant l'orderId)
    window.location = "./confirmation.html?orderId=" + orderInfo.orderId;
  })
  .catch (function(errorJson){console.log(errorJson)})
})
.catch(function(error){console.log(error)})
})
```

Redirection sur la page Confirmation

- Rediriger l'utilisateur sur la page Confirmation, en passant l'id de commande dans l'URL, dans le but d'afficher le numéro de commande.
- Sur cette page, l'utilisateur doit voir s'afficher son numéro de commande.
 - Il faudra veiller à ce que ce numéro ne soit stocké nulle part.

```
.then (function(response){  
    response.json()  
    .then(function(orderInfo){  
        // rediriger vers page confirmation avec parametre dans l'URL (en ajoutant l'ID de la cde dedans).  
        window.location = "./confirmation.html?orderId=" + orderInfo.orderId;  
    })  
    .catch (function(errorJson){console.log(errorJson)})  
})  
    .catch(function(error){console.log(error)})  
})  
})
```

```
front > js > JS confirmation.js > ...  
1  const orderLab=document.getElementById("orderId");  
2  
3  const urlParams = (new URL(location)).searchParams;  
4  const orderId = urlParams.get("orderId");  
5  
6  orderLab.innerHTML=orderId;  
7  localStorage.clear();
```





Plan de test



Merci



Q&R

